r/MachineLearning • 6y ago
amil123123

[D] Positional Encoding in Transformer

Discussion

Hey all,

I was reading up the transformer paper https://arxiv.org/abs/1706.03762. This architecture uses positional encoding which the attention layers ignore.

I don't understand two things -

1. Why use Sin & Cos as positional embeddings , why not any other function?
2. They also talk about training these positional embeddings, how do you go about training such embeddings. As in how do you let the model know that these embeddings are for the position

Thanks !

53 31 Share

Add your reply

Sort by: Best

mikeross0 • 6y ago

The thing that always gets me with positional embeddings is that it is preferable to add them to the word embeddings instead of concatenating them. We already know the dimensions of the word embeddings are related to semantics. So why embed position into the semantic space instead of adding additional dimensions to represent position? If someone can provide a good intuitive explanation for that, you will get all my internet points for the day!

21 Reply

pappypapaya • 6y ago • Edited 6y ago

In attention, we basically take two word embeddings (x and y), pass one through a Query transformation matrix (Q) and the second through a Key transformation matrix (K), and compare how similar the resulting query and key vectors are by their dot product. So, basically, we want the dot product between Qx and Ky, which we write as:

(Qx)'(Ky) = x' (Q'Ky). So equivalently we just need to learn one joint Query-Key transformation (Q'K) that transform the secondary inputs y into a new space in which we can compare x.

By adding positional encodings e and f to x and y, respectively, we essentially change the dot product to

which ask the additional questions, "how much attention should we pay to word x given the position f of word y", "how much attention should we pay to y given the position e of word x", and "how much attention should we pay to the position e of word x given the position f of word y".

Essentially, the learned transformation matrix Q'K with positional encodings has to do all four of these tasks simultaneously. This is the part that may appear inefficient, since intuitively, there should be a trade-off in the ability of Q'K to do four tasks simultaneously and well.

HOWEVER, MY GUESS is that there isn't actually a trade-off when we force Q'K to do all four of these tasks, because of some approximate orthogonality condition that is satisfied of in high dimensions. The intuition for this is that randomly chosen vectors in high dimensions are almost always approximately orthogonal. There's no reason to think that the word vectors and position encoding vectors are related in any way. If the word embeddings form a smaller dimensional subspace and the positional encodings form another smaller dimensional subspace, then perhaps the two subspaces themselves are approximately orthogonal, so presumably these subspaces can be transformed approx. independently through the same learned Q'K transformation (since they basically exist on different axes in high dimensional space). I don't know if this is true, but it seems intuitively possible.

If true, this would explain why adding positional encodings, instead of concatenation, is essentially fine. Concatenation would ensure that the positional dimensions are orthogonal to the word dimensions, but my guess is that, because these embedding spaces are so high dimensional, you can get approximate orthogonality for free even when adding, without the costs of concatenation (many more parameters to learn). Adding layers would only help with this, by allowing for nonlinearities.

We also ultimately want e and f to behave in some nice ways, so that there's some kind of "closeness" in the vector representation with respect to small changes in positions. The sin and cos representation is nice since nearby positions have high similarity in their positional encodings, which may make it easier to learn transformations that "preserve" this desired closeness.

(Maybe I'm wrong, and the approximate orthogonality arises from stacking multiple layers or non-linearities in the fully-connected parts of the transformer).

tl;dr: It is intuitively possible that, in high dimensions, the word vectors form a smaller dimensional subspace within the full embedding space, and the positional vectors form a different smaller dimensional subspace approximately orthogonal to the one spanned by word vectors. Thus despite vector addition, the two subspaces can be manipulated essentially independently of each other by some single learned transformation. Thus, concatenation doesn't add much, but greatly increases cost in terms of parameters to learn.

123 Reply ···

trajo123 • 1y ago

Isn't the attention (Qx)(Ky)' , so the K is transposed rather than Q? So:

(Q(x+e)) (K(y+f))' = (Qx+Qe) (Ky+Kf)' = (Qx) (Ky)' + (Qx) (Kf)' + (Qe) (Ky)' + (Qe) (Kf)' = Qxy'K' + Qxf'K' + Qey'K' + Qef'K'

This kind of breaks the whole discussion about Q'K, no?

amil123123 OP • 6y ago

Wow , that's one hell of an amazing explanation :)

12 Reply ···

slcyz • 6y ago

> respectively, we essentially change the dot product to

Man... that's fantastic. Even the professor of my deep learning course failed to explain that.

7 Reply ···

bergqvisten • 2y ago

Found this gem 4 years later trying to deepen my understanding of transformers... Wow, fantastic explanation!

6 Reply ···

pappypapaya • 2y ago

3 more replies

Sinkencronge • 6y ago • Edited 6y ago

1. To build up the intuition about it you may want to look how the proposed function and pairwise distance between position-vectors behave.

https://imgur.com/kpW5n4p

https://imgur.com/kaADdQB

2. Backpropagation.

6 Reply ···

amil123123 OP • 6y ago

Thanks for the response. I still have difficulty understanding about 1.
So the first image seems good in explaining the position however what does the second image denote.

Is it just because this function seemed to work well , that we went with it ?

2 Reply ···

Sinkencronge • 6y ago

The second image just shows Euclidean distances between the added embedding for a given position.

In the paper they write:

> We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed off set k,PE_pos+k can be represented as a linear function of PE_pos.

Unfortunately, I don't really wave a better way to elaborate it further in my mind at the moment. I'm sorry for that.

Answering your question, I personally think that they seem to simply pick up the first elegant solution to solve this problem.

However, I believe that there are a lot of more interesting ways to take advantage of positional encoding trick. I'm currently working on it for my own dataset.

4 Reply ···

amil123123 OP • 6y ago

Thanks for the explanation, it was good !

1 Reply ···

InsideAndOut • 6y ago

1. Because the distances between the neighboring values of the sine/cosine positional embeddings are "nice" (symmetrical, and decay sensibly with distance). You could come up with another function that does something similar.
2. For trained positional embeddings, you simply have another embedding matrix (one is for words, the other is for positions). For each position (up to `max_length`), you learn an embedding of the same dimension as the word embedding.

3 Reply ···

amil123123 OP • 6y ago

Thanks for the response!

What do you mean by symmetrical and decay sensibly ?
(Sorry , but I am still a newbie in this)

3 Reply ···

Sinkencronge • 6y ago

"Symmetrical" - see image 2.

"Decay sensibly with distance" - In this gif you can see entries in the distance matrix which have similar value [https://imgur.com/a/p7NcUm1]

White dots are points (corresponding to different pairwise positions) which lay in some epsilon neighborhood from some threshold tau which changes over time from 0 to maximum distance.

Hi, I'm a bot for linking direct images of albums with only 1 image

https://i.imgur.com/1jwYWbG.gifv

Source | Why? | Creator | ignoreme | deletthis

1 Reply ···

amil123123 OP • 6y ago

Ahh understood, thanks for the explanation!

1 Reply ···

MasterSama • 2y ago • Edited 2y ago

thanks a lot, I know its 4 years ago, but do you remember or could you kindly share the snippet that was used to visualize that image?
Edit:
I guess I figured it out!
this should be sth like this right?

```python

def plot_pe_dot_product_heatmap(positional_encoding):
dot_product = np.dot(positional_encoding, positional_encoding.T)
sns.heatmap(dot_product, cmap='Blues')
plt.ylabel('Position')
plt.xlabel('Position')
plt.show()

```

1 Reply ···

gdiamos • 6y ago

Papers usually post their final solution, not the path they took to get there. The sin/cos positional embeddings often seem unintuitive for this reason.

I find it helps to try to reconstruct the path.

1. Why do transformers need positional embeddings at all? What would happen if you removed them completely? Why?
2. Why not use simpler and more obvious positional embeddings like literally the position index?

2 Reply ···

Jean-Porte • 6y ago
Researcher

Regarding to sin and cos representation, my hypothesis is that together, they fully characterize a position

2 Reply ···

No-Theory-6868 • 4y ago

It seems like PE isn't always nesecary, and indeed might even harm the results:

"We find that removing the positional encoding even slightly improves the performance of these models"

https://stackoverflow.com/questions/61440281/is-positional-encoding-necessary-for-transformer-in-language-modeling

https://arxiv.org/abs/1905.04226

2 Reply ···

Great-Reception447 • 8mo ago

Reddit Rules Privacy Policy User Agreement Accessibility Reddit, Inc. © 2025. All rights reserved.